

# Route Hijacking and DoS in Off-Chain Networks

Analisi del Paper (ACM AFT 2020)

Autori: Saar Tochner, Aviv Zohar (HUJI), Stefan Schmid (Univ. of Vienna)

Salvucci Franco

Mat. *0368692*

29 settembre 2026

# Roadmap

- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 Punti di Forza

# Roadmap

- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 Punti di Forza

# 1. Contesto principale: Scalabilità Blockchain e Reti Off-Chain

- **Il Limite di Scalabilità On-Chain:** I sistemi blockchain classici (es. Bitcoin) gestiscono solo poche decine di transazioni al secondo (tps), a fronte delle migliaia gestite dai circuiti tradizionali (es. VisaNet).
- **Payment Channel Networks (PCN):** Reti peer-to-peer off-chain (come Lightning Network) create per scalare: spostano i pagamenti fuori catena tramite canali bidirezionali pre-finanziati, azzerando la latenza globale e riducendo le fee di consenso.
- **Multi-Hop Routing e HTLC:** Quando due nodi non hanno un canale diretto, i pagamenti sono concatenati su percorsi intermedi protetti da *Hashed Time-Locked Contracts* (HTLC), che impediscono il furto di fondi da parte dei nodi intermedi.
- **Source Routing:** Il mittente seleziona autonomamente l'intero percorso sulla base della topologia e dei parametri di costo ricevuti tramite protocollo di gossip.
- **Trade-off Sicurezza-Privacy:** L'uso dell'onion routing garantisce la privacy ma scherma la tracciabilità: in caso di fallimento, il mittente non può individuare quale nodo intermedio sia responsabile del drop.

# Roadmap

- 1 Contesto
- 2 **Tesi principale**
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 Punti di Forza

## 2. Tesi Principale e Trade-off Economico

### Tesi dello Studio

Un attaccante esterno può, con un investimento monetario irrisorio ( $< \$16$ ) e aprendo appena 5 **nuovi** canali, dirottare circa il 65% delle rotte dell'intera rete Lightning, mentre con appena 30 canali dirottare circa l'80% delle rotte ed eseguire un Denial-of-Service sistematico a danno degli utenti.

### Innovazione e Trade-off Fondamentale del Difensore:

- **Oltre gli attacchi tradizionali:** Supera i modelli classici (congestione dei canali o isolamento topologico), sfruttando direttamente gli algoritmi di shortest-path per attirare il traffico.
- **Costo della Difesa:** Per rendersi immuni all'hijacking, gli utenti dovrebbero accettare percorsi deliberatamente più cari, distruggendo la pressione economica concorrenziale che rende la rete conveniente.

# Roadmap

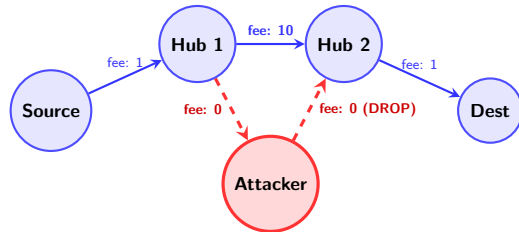
- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco**
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 Punti di Forza

## 3.1 Il Meccanismo: Route Hijacking & Black-Hole DoS

### Dinamica dell'Attacco:

- 1 L'avversario apre canali verso nodi cardine ad alta centralità.
- 2 Annuncia pubblicamente **zero commissioni** (base fee = 0, prop fee = 0) e ritardi minimi.
- 3 Gli algoritmi di source routing scelgono il canale malevolo come scorciatoia a costo minimo.
- 4 Ricevuto il pacchetto onion HTLC, l'attaccante **scarta il payload** (*black hole*) o interrompe la catena.

Il pagamento fallisce e il mittente non sa a chi attribuire il fallimento a causa dell'onion routing.



### Vulnerabilità Riconfigurabile

Per default tutti i nodi accettano proposte di canale in ingresso: l'attaccante può collegarsi a chiunque senza autorizzazione preventiva.

## 3.2 Amplificazione dell'Attacco: Congelamento tramite Delay

L'attaccante può convertire un DoS istantaneo in un grave **blocco di liquidità**.

### Meccanismo degli HTLC Decrementali:

- Per impedire che un nodo intermedio paghi senza aver ricevuto il rimborso, i timeout HTLC devono decrescere:

$$T_{\text{in}} \geq T_{\text{out}} + \Delta_{\text{delay}}$$

- L'attaccante non scarta subito il pacchetto: finge di elaborarlo e trattiene la transazione fino alla scadenza del timeout.
- **Effetto Congelamento:** I fondi di tutti i nodi precedenti restano immobilizzati nel canale per centinaia di blocchi (fino a 144 blocchi  $\approx$  24 ore).

### Trade-off dell'Attaccante

#### Dilemma Ritardi vs Appetibilità:

- Dichiarare un  $\Delta_{\text{delay}}$  elevato rende l'attacco più dannoso per la liquidità altrui.
- Tuttavia, aumenta il peso computato dagli algoritmi di routing, riducendo le coppie catturate.
- Offrendo **fee pari a zero**, l'attaccante compensa quasi per intero l'aggravio di ritardo.

# Roadmap

- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto**
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 Punti di Forza

## 4.1 Implementazione Ind (Golang)

### Funzione di Costo Canale (Dijkstra):

$$\text{fee}_i = \text{ams}[i + 1] \cdot p[i].\text{propFee} + p[i].\text{baseFee}$$

$$w[i] = \text{ams}[i + 1] \cdot p[i].\text{delay} \cdot r_f + \text{fee}_i$$

- Fattore di rischio ritardo:  $r_f = 15 \times 10^{-9}$ .

### Penalità Mission Control:

- Aggiunge  $\frac{100}{P(h)}$  al peso dei canali falliti.
- Decadimento su  $h$  (ore dall'errore):

$$P(h) = 0.6 - \frac{0.6}{2^h} \quad (P = 0 \text{ per } h < 1)$$

### Vulnerabilità ed Efficacia

- **Penalità cieca (Onion):** la sorgente non sa quale nodo abbia fallito e penalizza tutti gli hop della rotta; isola canali onesti se l'accesso all'hub è obbligato.
- **Dominanza fee zero:** con default  $\text{baseFee} = 1000 \text{ msat}$ , impostare  $\text{fee} = 0$  azzerava il costo e assorbe l'intera penalità di Mission Control ( $\approx 400$ ).
- **Bypass del fallimento:** trattenendo l'HTLC fino all'ultimo istante, massimizza il DoS senza registrare subito un errore.

## 4.2 Implementazione c-lightning (C)

### Weight Fuzzing Stocastico:

- Perturba casualmente **solo la fee**:

$$\text{scale} = 1 + \text{fuzz} \cdot \left( \frac{2^h}{2^{64}-1} - 1 \right)$$

con  $h = \text{SipHash-24}$  e  $\text{fuzz} = \pm 0.05$  ( $\pm 5\%$ ).

- La fee perturbata risulta:

$$\text{fee}_{\text{fuzz}} = \text{scale} \cdot \text{fee}_i$$

### Peso Finale con Delay Penalty:

$$w[i] = (\text{ams}_{i+1} + \text{fee}_{\text{fuzz}}) \cdot (\text{delay}_i \cdot r_f) + 1$$

- Parametro di rischio di default:  $r_f = 10$ .

### Criticità del Fuzzing sulle Fee

- Collasso su fee nulle:** poiché la perturbazione agisce da moltiplicatore:

$$\text{fee} = 0 \implies \text{scale} \cdot 0 = 0$$

L'annullamento delle fee azzerava l'entropia del routing; il percorso dell'attaccante resta il migliore in modo **deterministico**.

- Inefficacia empirica:** test con fuzzing elevato ( $\pm 30\%$ ) dimostrano che le scelte di routing sui nodi centrali non variano.

## 4.3 Implementazione Eclair (Scala)

### Ponderazione Multivariata Canale:

$$w[i] = \text{fee}_i \cdot (\alpha_d \hat{d} + \alpha_c \hat{c} + \alpha_h \hat{h})$$

- $\hat{d}$ : ritardo norm. ( $9 < d < 2016$ ,  $\alpha_d = 0.15$ )
- $\hat{c}$ : capacità norm. ( $10^3 < c < 2^{24}$ ,  $\alpha_c = 0.50$ )
- $\hat{h}$ : età norm. ( $0 < h < 8640$ ,  $\alpha_h = 0.35$ )

### Top-3 Randomization:

- Selezione uniforme tra le **3 rotte migliori**.

### Perché Eclair è Vulnerabile

- **Accoppiamento moltiplicativo:** la fee moltiplica l'intera parentesi: dimezzando la tariffa e raddoppiando il delay il peso resta invariato, aprendo ad attacchi DoS con ritardo prolungato.
- **Monopolio delle top 3:** offrendo scorciatoie a costo zero, l'attaccante colloca facilmente i propri canali in **tutte e 3 le rotte migliori**, rendendo inefficace la randomizzazione.

# Roadmap

- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete**
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 Punti di Forza

## 5. Setup Sperimentale e Topologia di Rete

### Dati di Misurazione (Mainnet):

- Snapshot estratti interrogando un nodo Ind di produzione con il comando `lncli describegraph`.
- Topologia reale:  $\sim$  **4.300** nodi,  $\sim$  **33.600** canali, capacità media di 0.028 BTC.
- Dati raccolti e verificati su 5 snapshot distinti tra novembre 2018 e maggio 2020.

### Insight Comportamentali Critici:

- **Dominio dei Default:** Il 60% dei canali usa la configurazione di default (base fee = 1000 msat, prop fee = 1/1000).
- **Altruismo diffuso:** La seconda configurazione più comune ha fee = 0. Nessuna tendenza a massimizzare i profitti.

### Iper-centralizzazione

- Il **25%** dei nodi ha grado 1 (foglie).
- Il **14%** ha grado 2.
- Circa l'**88%** di tutti i canali è collegato a un hub con grado  $> 600$ .

La rete presenta una forte topologia *scale-free*, rendendola vulnerabile a pochi attacchi mirati.

# Roadmap

- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali**
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 Punti di Forza

## 6.1 Risultati: Vulnerabilità a Nodi Collusi

Cosa accade se un piccolo gruppo di nodi centrali preesistenti decide di colludere per fare DoS?

### Centralità Cumulativa del Traffico:

- **5 nodi collusi** sono sufficienti per intercettare e bloccare circa il **60%** di tutte le rotte dell'intera rete.
- **10 nodi** controllano l'**80%** dei flussi di pagamento.
- **30 nodi** intercettano oltre il **95%** di tutte le transazioni possibili tra coppie di nodi.

I risultati sono quasi sovrapponibili tra lnd, c-lightning ed Eclair.

### Fallimento del Top-3 di Eclair

Confrontando le metriche su Eclair:

- *Best path*: 65% dirottato.
- *Average (top-3)*: 67% dirottato.
- *Worst (tutte e 3)*: 60% dirottato.

La randomizzazione su 3 percorsi non protegge, poiché gli hub centrali sono colli di bottiglia su tutte le alternative.

## 6.2 Risultati: Attaccante Esterno a Basso Costo

Un partecipante esterno si connette alla rete e apre  $k$  nuovi canali strategici con fee nulle.

### Efficacia dell'Iniezione di Link:

- Con soli **5 nuovi canali**, l'attaccante dirotta:
  - $\sim 65\%$  su Eclair
  - $\sim 70\%$  su c-lightning
  - $\sim 75\%$  su Ind
- Con **30 nuovi canali**, l'attaccante cattura oltre l'**80%–85%** di tutte le transazioni globali della rete.
- L'apertura dei canali crea perfino nuove connettività prima inesistenti, rendendo l'attaccante ancora più appetibile.

### Costi Economici Reali (Maggio 2020)

- Costo per canale:  $\approx$  **\$3.10** (on-chain fee + liquidità minima vincolata).
- **Attacco a 5 canali (70% DoS):**

Costo Totale  $<$  **\$16**

- **Attacco a 30 canali (85% DoS):**

Costo Totale  $<$  **\$100**

La liquidità vincolata non viene spesa ma recuperata a fine attacco chiudendo il canale.

## 6.3 Inefficacia delle Difese Native dei Client

Gli autori hanno verificato empiricamente l'efficacia dei meccanismi difensivi già integrati:

- **Weight Fuzzing di c-lightning ( $\pm 5\%$ ,  $\pm 15\%$ ,  $\pm 30\%$ ):**
  - Il disturbo moltiplica la sola fee del canale.
  - Poiché l'attaccante offre commissioni nulle (0 msat), moltiplicare zero per qualsiasi fattore di fuzzing produce sempre zero: il canale malevolo resta la scelta dominante.
- **Penalizzazione dei Fallimenti di Ind (Mission Control):**
  - Ind penalizza i canali falliti aggiungendo un peso legato al tempo trascorso:  $\frac{n}{0.6 - 0.6/2^h}$ .
  - La massima penalità applicabile ( $\approx 400$ ) è nettamente inferiore al risparmio offerto dalla base fee a 0 rispetto al valore standard di 1000 msat.
  - A causa dell'onion routing, il mittente penalizza indistintamente l'intero prefisso della rotta.
- **Attacco Agnostico basato sul Grado (Degree Centrality):**
  - Collegandosi semplicemente ai nodi con grado più elevato, l'attaccante ottiene percentuali di successo quasi identiche senza dover conoscere quale client usino le vittime.

## 6.4 Robustezza e Validazione Storica (2018–2020)

Per validare la persistenza della vulnerabilità, gli autori hanno analizzato 5 snapshot temporali:

| Snapshot Temporale | Nodi Pubblici | Canali Pubblici | Hijack (5 link) | Hijack (30 link) |
|--------------------|---------------|-----------------|-----------------|------------------|
| 9 Novembre 2018    | ~ 2.100       | ~ 10.500        | ~ 68%           | ~ 82%            |
| 27 Febbraio 2019   | ~ 3.200       | ~ 21.000        | ~ 72%           | ~ 86%            |
| 27 Luglio 2019     | ~ 4.000       | ~ 31.000        | ~ 58%           | ~ 81%            |
| 14 Dicembre 2019   | ~ 4.150       | ~ 32.500        | ~ 59%           | ~ 85%            |
| 20 Maggio 2020     | ~ 4.300       | ~ 33.600        | ~ 66%           | ~ 84%            |

**Conclusione della validazione:** Non si tratta di una vulnerabilità transitoria legata a una specifica fase embrionale della rete, bensì di un difetto strutturale che permane e si consolida con la crescita del network.

# Roadmap

- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante**
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 Punti di Forza

## 7.1 Modellazione del Problema e Complessità

### Modellazione della Rete:

- Grafo dei canali  $G = (V, E)$ , dove  $V$  sono i nodi e  $E$  i canali di pagamento bidirezionali.
- Algoritmo di routing  $\mathcal{A}_W(s, t, G)$ : seleziona il cammino valido da  $s$  a  $t$  in base ai pesi  $W$ .

### Centralità di un Insieme di Canali:

- Sia  $S \subseteq E$  un insieme di canali: la sua **centralità**  $C(S)$ :  $2^E \rightarrow [0, 1]$  è la frazione di rotte della rete che transitano attraverso *almeno uno* dei canali in  $S$ .
- **Nota fondamentale:**  $C(S)$  non è la semplice somma delle centralità dei singoli archi, poiché più canali possono coprire la medesima rotta  $(s, t)$ .

### Obiettivo e Complessità Computazionale

- **Problema dell'Attaccante:** Dato un nodo malevolo  $\bar{v}$  e un budget di  $k$  canali ad offerta zero (fee = 0, delay = 0), trovare il sottoinsieme di collegamenti  $S^* \subset V$  con  $|S^*| \leq k$  che massimizzi  $C(S)$ .
- **NP-Completezza:** Trovare la combinazione ottima di  $k$  archi per massimizzare la centralità è un problema **NP-hard** (riduzione dal problema di *Betweenness Centrality Maximization with Bounded Budget*).
- La ricerca esaustiva  $\binom{|V|}{k}$  è intrattabile sulla rete reale ( $|V| > 4.300$ ).

## 7.2 Submodularità e Garanzie dell'Algoritmo Greedy

### Proprietà di Submodularità (Lemmi 1 e 2):

- La funzione di centralità  $C(\cdot)$  è non-negativa, monotona e **submodulare**:

$$C(A \cup \{e\}) - C(A) \geq C(B \cup \{e\}) - C(B)$$

per ogni coppia di insiemi  $A \subseteq B \subseteq E$ .

- Intuizione:** L'aggiunta di un nuovo canale garantisce un guadagno marginale decrescente man mano che l'attaccante ha già monopolizzato porzioni del grafo.

### Algoritmo Greedy:

- Inizializza  $E_{\text{att}} = \emptyset$ .
- Per  $i = 1, \dots, k$ , sceglie l'arco  $e^*$  che massimizza l'incremento marginale:  
$$e^* = \arg \max_{v \in V} C(E_{\text{att}} \cup \{(\bar{v}, v)\})$$

### Teorema 1: Garanzia di Approssimazione

**Formulazione (Nemhauser et al. 1978):** Applicando l'euristica greedy su una funzione d'insieme monotona e submodulare con vincolo di cardinalità  $k$ , si ottiene un fattore di approssimazione analitico:

$$1 - \left(1 - \frac{1}{k}\right)^k \geq 1 - \frac{1}{e} \approx 63.2\%$$

### Riscontro Empirico:

- Il 63.2% rappresenta un limite inferiore teorico nel caso peggiore.
- Sulla topologia reale di Lightning, l'euristica greedy cattura il **65%–75%** del traffico con soli  $k = 5$  canali e oltre l'**80%** con  $k = 30$ .

## 7.3 Implementazione Efficiente dell'Attacco

### Algoritmo 2: Programmazione Dinamica

**Problema:** ricalcolare l'intero routing per ogni candidato  $v \in V$  costa  $O(|V|^3)$ .

**Risoluzione via Preprocessing (DP):**

- 1 Esegue Dijkstra *una tantum* per ogni  $s \in V$ , salvando in tabella le distanze minime ottime  $\text{dist}(s, t)$ .
- 2 Per ogni candidato  $v$ , verifica in tempo  $O(1)$  la condizione di deviazione:

$$\text{dist}(s, v) + \text{dist}(\bar{v}, t) \leq \text{dist}(s, t)$$

- 3 Incrementa il contatore se il cammino alternativo attraverso  $\bar{v}$  costa meno.

### Algoritmo 3: Euristiche Scalabili (Paper)

**Ottimizzazioni per la Rete Reale:**

- **Short-Circuiting Rotte Dirottate:** Salta immediatamente le coppie  $(s, t)$  già catturate nei round greedy precedenti, riducendo drasticamente le operazioni di controllo.
- **Filtraggio Topologico:** Itera solo sui nodi sorgente che possiedono già un cammino valido verso il candidato  $v$ .
- **Inclusione Limiti Protocollo:** Scarta a priori percorsi che superano i default di client reali (`max_hops`, soglie massime di commissione).

*Risultato:* Esecuzione completa in pochi minuti su grafi di oltre 33.000 canali.

# Roadmap

- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti**
- 9 Punti di Forza

# 8.1 Contromisure: La Nuova Policy di Routing

## I 4 Principi Guida per un Routing Resiliente:

- **Disaccoppiamento Additivo del Delay:** Il ritardo non deve moltiplicare la fee; va inserito come addendo proporzionale al valore trasferito.
- **Fuzzing sul Peso Totale:** Il disturbo stocastico ( $\mathcal{N}(1, \sigma)$ ) deve agire sull'intero costo del canale, non solo sulla fee (evita  $\text{fuzz} \times 0 = 0$ ).
- **Premialità per Canali Puri e Maturi:** Penalizzare canali a bassa capacità e privilegiare link con anzianità elevata (costo opportunità di liquidità bloccata).
- **Superamento del Top- $k$ :** Eliminare la scelta casuale uniforme sulle prime  $k$  rotte per impedire il monopolio da shortcut.

### Funzione di Costo Proposta dal Paper

Adattamento della metrica di Eclair con perturbazione globale gaussiana ( $\sigma = 0.2$ ):

$$w[i] = \text{scale} \cdot \left( \alpha_d \hat{d} + \alpha_h \hat{h} - \alpha_c \hat{c} - \beta_{\text{int}} \cdot (\text{cap} \cdot \text{height}) + \frac{\text{fee}_i}{\text{ams}} \cdot \gamma_f \right)$$

Parametri calibrati:  $\alpha_d = 0.5$ ,  $\alpha_h = 0.5$ ,  $\alpha_c = 0.3$ ,  $\gamma_f = 100$ .

### Effetto Empirico (Fig. 21–23):

- Riduce la centralità dei primi 5 nodi dal 60% al 45%.
- Abbassa il successo dell'attacco delay ad alti blocchi ( $> 144$ ) a percentuali prossime allo zero.

## 8.2 Sintesi dell'Efficacia delle Contromisure

| Meccanismo             |                | Resilienza Base DoS                                                                   | Resilienza Delay Attack                                                     | Trade-off / Limite                                                  |
|------------------------|----------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Fuzzing Fee</b>     | (c-lightning)  | <b>Nulla:</b> con fee = 0, scale · 0 = 0; rotta deterministica.                       | <b>Bassa:</b> il disturbo sulla fee non compensa il blocco temporale.       | Costo nullo; vulnerabile a scorciatoie a tariffa zero.              |
| <b>Top-3 Rotte</b>     | (Eclair)       | <b>Nulla:</b> scorciatoie gratuite monopolizzano le prime 3 posizioni.                | <b>Pessima:</b> la fee moltiplica il delay; fee minime azzerano il ritardo. | Ripartisce il carico; annullata da link avversari multipli.         |
| <b>Mission Control</b> | (Ind)          | <b>Insufficiente:</b> onion routing maschera il colpevole; risparmio fee > penalità.  | <b>Nulla:</b> l'attesa fino al timeout non produce errore immediato.        | Falsi positivi: penalizza nodi onesti lungo lo stesso percorso.     |
| <b>Policy Proposta</b> | (Paper Sez. 5) | <b>Alta:</b> rumore $\mathcal{N}(1, \sigma)$ sul costo totale e peso per nodi maturi. | <b>Molto Alta:</b> delay additivo; successo crolla sopra 144 blocchi.       | <b>Trade-off:</b> commissioni leggermente maggiori per nodi solidi. |

**Il Trade-Off Fondamentale del Difensore:** La ricerca assoluta del costo minimo espone sistematicamente al dirottamento dei percorsi. Per proteggere la rete è necessario accettare tariffe medie leggermente superiori e privilegiare nodi maturi ad alta liquidità.

# Roadmap

- 1 Contesto
- 2 Tesi principale
- 3 Meccanismo dell'Attacco
- 4 Implementazioni a Confronto
  - Ind (Golang)
  - C-lightning (C)
  - Eclair (Scala)
- 5 Setup Sperimentale & Topologia di Rete
- 6 Risultati Sperimentali
- 7 Strategia Ottima dell'Attaccante
  - Modellazione Formale del problema
  - Submodularità & Algoritmo Greedy
  - Implementazioni
- 8 Contromisure e Miglioramenti
- 9 **Punti di Forza**

## 9. Punti di Forza del Paper

- **Rigore Matematico e Algoritmico:** Dimostrazione formale della submodularità della centralità e derivazione del limite di approssimazione greedy  $1 - 1/e$ .
- **Validazione su Dati Reali di Produzione:** Misurazioni empiriche condotte direttamente sulla mainnet di Bitcoin per un arco temporale di 13 mesi.
- **Analisi Comparativa tra Codebase Reali:** Studio dettagliato delle differenze architetturali e algoritmiche tra le tre implementazioni dominanti (*Ind*, *c-lightning*, *Eclair*).
- **Dimostrazione dell'Economicità Estrema:** Prova tangibile che un Denial-of-Service di portata sistemica ha un costo irrisorio ( $< \$16$ ).

## Articolo di Riferimento:

- Tochner, S., Zohar, A., & Schmid, S. (2020). *Route Hijacking and DoS in Off-Chain Networks*. In ACM Conference on Advances in Financial Technologies (AFT '20), New York, USA.

## Riferimenti Correlati:

- Poon, J., & Dryja, T. (2016). *The Bitcoin Lightning Network: Scalable off-chain instant payments*.
- Nemhauser, G. L., Wolsey, L. A., & Fisher, M. L. (1978). *An analysis of approximations for maximizing submodular set functions*. Mathematical Programming.
- Bergamini, E., Crescenzi, P., et al. (2018). *Improving the betweenness centrality of a node by adding links*. Journal of Experimental Algorithmics.
- Rohrer, E., Malliaris, J., & Tschorsch, F. (2019). *Discharged payment channels: Quantifying the Lightning Network's resilience to topology-based attacks*.